



Continuous Simulation and Input Modelling

Start With a Room You Know

- The server room is 22 degrees when the shift begins
- The servers release heat every minute
- The air conditioner removes more heat when the room is hotter
- The operating limit is 27 degrees
- Will the room cross the limit, and when?
- A single temperature reading cannot answer either question

Why Guessing Fails Here

- Server heat pushes the temperature upward
- Cooling pulls the temperature toward the supply-air temperature
- The cooling effect changes whenever the room temperature changes
- The two effects act at the same time
- The path matters because the limit can be crossed before equilibrium
- We need to follow the state continuously through time

What You Will Learn Today

- The definition of continuous simulation
- How a rate equation describes changing state
- Euler's method, worked by hand and in Python
- Why step size affects accuracy and stability
- How Runge-Kutta improves the numerical solution
- How observed data becomes a defensible simulation input

Continuous Simulation: The Definition

- A continuous model represents state that can vary at every instant
- A rate equation states how fast the state changes now
- The current state affects what happens next
- Numerical integration builds the state trajectory over time
- We experiment on the model by changing inputs or parameters
- The output is a path through time, not one isolated value

Where We Left Off: Discrete Events

- A discrete-event model jumps from one state-changing event to the next
- Between events its state remains unchanged
- A continuous model changes state even when no event occurs
- Both models have a clock, state, inputs, and outputs
- Both require verification, validation, and a planned experiment
- Only the state-update mechanism changes today

Discrete and Continuous Models

- Queue length changes by whole customers at event times
- Temperature can change by any amount at any instant
- A discrete-event model schedules arrivals and departures
- A continuous model evaluates rates and integrates them over time
- Some systems combine events with continuous physical state
- The decision question determines which behavior must be represented

State, Rate, and Time Step

- State is the information needed to describe the system now
- The rate of change says how quickly that state is moving
- A positive rate increases the state and a negative rate decreases it
- A time step is the interval between numerical updates
- The method assumes something about the rate during each step
- That assumption creates numerical error

The Server-Room Rate Equation

- T is the room temperature in degrees Celsius
- $dT/dt = -0.1(T - 18) + 1.2$ degrees per minute
- The cooling term is negative whenever T is above 18
- The servers add 1.2 degrees per minute
- At $T = 22$ the net rate is +0.8 degrees per minute
- The rate changes as soon as the temperature changes

The Standard Terms

Term	Formal meaning	Server-room example
State variable	Value describing the system now	Temperature T
Rate of change	Change in state per unit time	dT/dt in degrees/min
Initial condition	Starting state at time zero	$T(0) = 22$ degrees
Parameter	Fixed coefficient for one run	Cooling = $0.1/\text{min}$
Step size	Time between numerical updates	$h = 5$ minutes
Equilibrium	State where the net rate is zero	$T = 30$ degrees

Initial Conditions, Parameters, and Inputs

- The initial condition states where the trajectory begins
- A parameter keeps the same value during one run
- An external input may change as simulated time passes
- The server-room model begins with $T(0) = 22$ degrees
- Cooling strength and server heat are parameters in this version
- A changing workload could make server heat a time-varying input

Equilibrium and Feedback

- Equilibrium occurs when the net rate is zero
- Set $-0.1(T - 18) + 1.2$ equal to zero
- The equilibrium temperature is 30 degrees
- Above 30 the room cools and below 30 it warms
- This restoring behavior is negative feedback
- The operating limit is 27, so equilibrium alone already signals a problem

Numerical Integration

- Some rate equations have an exact formula and many do not
- A numerical method advances the state through finite steps
- Each step estimates the change from the current rate
- Adding the estimated change produces the next state
- Repeating the calculation builds the trajectory
- The chosen method and step size affect the answer

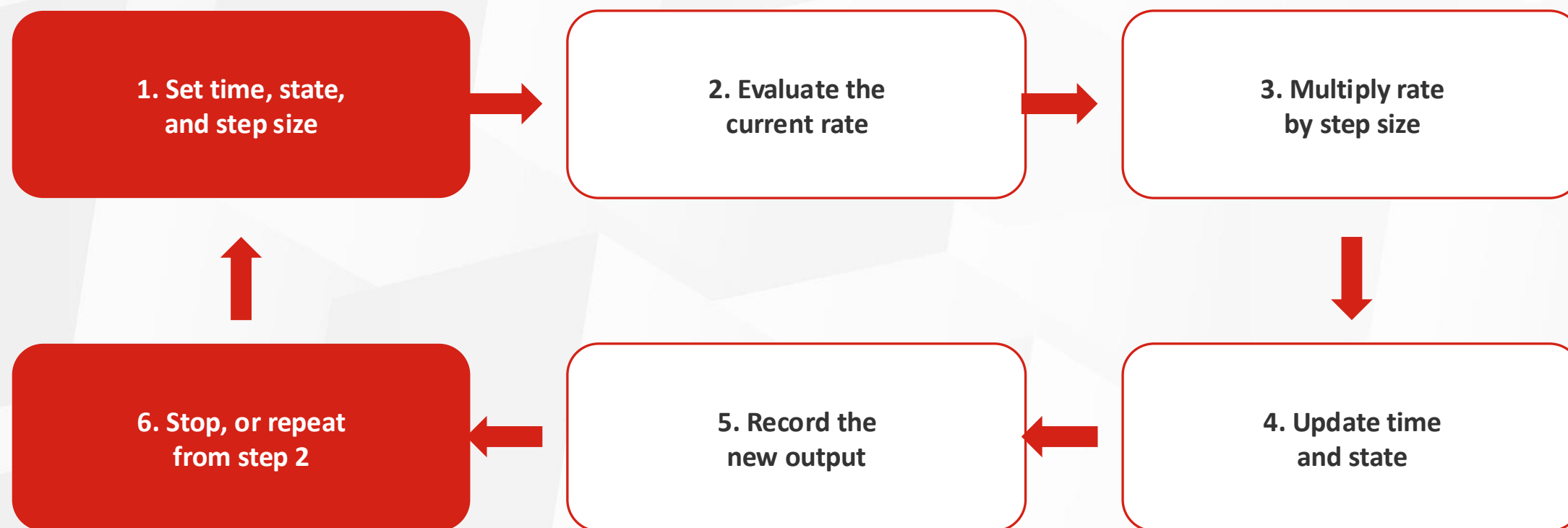
Euler's Method

- Evaluate the rate at the start of the step
- Assume that rate holds across the whole step
- Estimated change = step size times current rate
- New state = old state + estimated change
- Formula: $T_{\text{next}} = T_{\text{now}} + h \text{ times rate}(T_{\text{now}})$
- The method is simple enough to calculate by hand

One Euler Step

- Start at minute 0 with $T = 22$ degrees
- The current rate is $-0.1(22 - 18) + 1.2 = +0.8$
- Choose a step of $h = 5$ minutes
- Estimated change = 5 times $0.8 = 4$ degrees
- The next temperature is $22 + 4 = 26$ degrees
- At minute 5 the calculation begins again with the new rate

Euler's Method as an Algorithm



One Pass Through Euler's Method

- The current time is 5 minutes and T is 26 degrees
- The current rate is $-0.1(26 - 18) + 1.2 = +0.4$
- Across a 5 minute step the estimated change is +2 degrees
- The state updates from 26 to 28 degrees
- The clock advances from minute 5 to minute 10
- The smaller rate shows the room approaching equilibrium

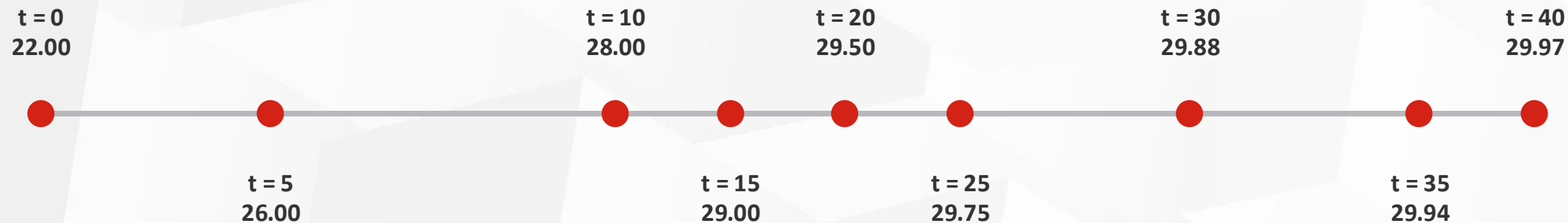
Euler by Hand, Steps of 5 Minutes

- Initial temperature: 22 degrees
- Air-conditioner temperature: 18 degrees
- Cooling strength: 0.1 per minute
- Server heat: 1.2 degrees per minute
- Euler step size: 5 minutes
- Run `demo 07_euler_by_hand.py` after this slide

The Euler Trace

Minute	Rate now	Euler T	Exact T	Absolute error
0	+0.80	22.00	22.00	0.00
5	+0.40	26.00	25.15	0.85
10	+0.20	28.00	27.06	0.94
15	+0.10	29.00	28.21	0.79
20	+0.05	29.50	28.92	0.58
25	+0.02	29.75	29.34	0.41
30	+0.01	29.88	29.60	0.27
35	+0.01	29.94	29.76	0.18
40	+0.00	29.97	29.85	0.12

How Simulated Time Advances



Euler updates the state at chosen time steps. The underlying model describes continuous change between them.

Reading the Euler Trace

- The temperature rises quickly at first
- The rate falls as stronger cooling offsets server heat
- The trajectory approaches 30 degrees without crossing it
- Euler stays above the exact solution for this model and step size
- The largest shown error is 0.94 degree at minute 10
- The operating limit of 27 degrees is crossed before minute 10

The Exact Solution Is a Check

- This particular equation has an exact solution
- $T(t) = 30 + (22 - 30) \exp(-0.1t)$
- At minute 20 the exact temperature is 28.917 degrees
- Euler with 5 minute steps gives 29.500 degrees
- Their difference measures numerical error for this test case
- Most practical models need other verification evidence because no exact answer exists

Step Size and Error

- A smaller step updates the rate more often
- Euler then spends less time using an outdated rate
- Halving the step roughly halves the error for this model
- That pattern is called first-order accuracy
- Smaller steps require more calculations
- The step is small enough when further reduction no longer changes the decision

The Halving Test

- Exact temperature at minute 20: 28.917 degrees
- $h = 5$ gives 29.500, error 0.583
- $h = 2.5$ gives 29.199, error 0.282
- $h = 1.25$ gives 29.055, error 0.138
- $h = 0.625$ gives 28.986, error 0.068
- Each halving makes the error about half as large

Accuracy and Stability

- Accuracy asks how close the numerical answer is to the true solution
- Stability asks whether numerical errors remain controlled
- A 5 minute step approaches 30 degrees smoothly
- A 20 minute step bounces between 22 and 38 forever
- A 25 minute step swings to impossible values and grows without bound
- The model is unchanged in every case; only the numerical step differs

Setup: Python and the Demos

```
python -m venv .venv
```

```
.venv\Scripts\activate      # Windows PowerShell  
source .venv/bin/activate   # macOS and Linux
```

```
pip install simpy
```

```
python 07_euler_by_hand.py
```

```
# Demos 07 to 11 use only the standard library
```

```
# Demo 12 uses SimPy
```

Mapping the Model to Python

- Constants store the initial condition and parameters
- `rate_of_change(T)` implements the differential equation
- The variable `h` stores the step size
- The loop repeats Euler's update
- The exact function supplies a verification benchmark
- The printed table keeps model output and numerical error together

Euler in Python

```
AIRCON = 18.0
COOLING = 0.1
SERVER_HEAT = 1.2

def rate_of_change(T):
    return -COOLING * (T - AIRCON) + SERVER_HEAT

h, T, t = 5.0, 22.0, 0.0
for step in range(8):
    T = T + h * rate_of_change(T)
    t = t + h
    print(t, round(T, 2))
```

What Demo 7 Shows

- The room settles near 30 degrees
- The 27 degree operating limit is exceeded
- The rate starts at +0.8 degree per minute and shrinks toward zero
- Euler assumes the starting rate holds for the whole step
- The true rate falls during the step, so Euler adds too much
- Changing the cooling strength changes both the trajectory and equilibrium

Testing Step Size and Stability

```
def euler(h, until):  
    T = 22.0  
    for step in range(int(round(until / h))):  
        T = T + h * rate_of_change(T)  
    return T  
  
for h in [5, 2.5, 1.25, 0.625]:  
    value = euler(h, 20)  
    print(h, value, abs(value - exact(20)))
```

Smaller Steps, Smaller Error

- At minute 20 the exact answer is 28.917 degrees
- A 5 minute step misses by 0.583 degree
- A 2.5 minute step misses by 0.282 degree
- A 1.25 minute step misses by 0.138 degree
- Ten times smaller steps are needed for about one more decimal place
- Accuracy has a computational cost

When the Step Is Too Big

```
step 5 min: 26.0, 28.0, 29.0, 29.5, 29.8, 29.9
step 15 min: 34.0, 28.0, 31.0, 29.5, 30.2, 29.9
step 20 min: 38.0, 22.0, 38.0, 22.0, 38.0, 22.0
step 25 min: 42.0, 12.0, 57.0, -10.5, 90.8, -61.1
```

```
# Same model, same parameters, different numerical step
# Accuracy means close to the true solution
# Stability means the numerical errors stay controlled
```

A Better Method: Runge-Kutta

- Euler looks at the rate once at the start of each step
- Fourth-order Runge-Kutta evaluates four rates
- It looks at the start, twice near the middle, and at the end
- A weighted average of the four rates estimates the change
- Each step costs more arithmetic but can be far more accurate
- Run demo 09_runge_kutta.py after this slide

Runge-Kutta in Python

```
rate_1 = rate_of_change(T)
rate_2 = rate_of_change(T + h * rate_1 / 2)
rate_3 = rate_of_change(T + h * rate_2 / 2)
rate_4 = rate_of_change(T + h * rate_3)

average_rate = (rate_1 + 2 * rate_2 +
                2 * rate_3 + rate_4) / 6

T = T + h * average_rate
```

Method Comparison at Minute 20

- Exact temperature: 28.917318 degrees
- Euler with $h = 5$ has error 0.582682
- Runge-Kutta with $h = 5$ has error 0.001716
- Euler with $h = 0.1$ uses 200 steps and has error 0.010845
- Runge-Kutta with $h = 5$ uses 4 steps and is still more accurate
- Compare methods on accuracy and effort, not step size alone

Why Runge-Kutta Wins

- The rate changes during a time step
- Euler ignores that change until the next step
- Runge-Kutta samples how the rate changes inside the step
- The weighted average follows a curved trajectory more closely
- A better method does not remove the need for a step-size check
- A solver can be numerically accurate while the model remains invalid

Where Simulation Inputs Come From

- Last session the queue used an assumed 1.5 minute service time
- A real model should use observations from the process
- Input modelling turns those observations into a sampling rule
- The rule must preserve the range, shape, and variability that matter
- A mean alone hides the slow cases that create queues
- The choice must be documented and tested before use

The Five Steps of Input Modelling

- 1. Collect representative observations from the process
- 2. Inspect the data before choosing a distribution
- 3. Choose a family whose support and shape make sense
- 4. Estimate the parameters from the observed sample
- 5. Assess whether the fitted model is adequate for its purpose
- Then test whether the choice changes the simulation decision

Step 1: Collect Representative Data

- Measure the quantity that the simulation will sample
- Record its unit, time period, location, and operating condition
- Include busy and quiet periods when both are in scope
- Do not remove long observations only because they look inconvenient
- Separate data errors from rare but real behavior
- The demo uses 200 service timings collected with a stopwatch

Steps 2 and 3: Inspect and Choose

- The sample mean is 1.50 minutes and the median is 0.95
- The longest service takes 10.86 minutes
- Many customers are quick and a few are very slow
- The histogram is tall on the left with a long right tail
- A symmetric Normal distribution cannot reproduce that shape well
- An Exponential distribution matches the positive, right-skewed pattern

Step 4: Estimate the Parameter

- The Exponential distribution has one scale parameter: its mean
- Estimate that parameter with the sample mean of 1.50 minutes
- The simulation can now call `random.expovariate(1 / fitted_mean)`
- A fitted sample has mean 1.48 and median 0.96 in demo 10
- Close summaries are useful evidence but they do not establish fit
- Run `demo 10_fit_a_distribution.py` after this slide

The Chi-Square Table for Our Data

Service-time bin	Observed	Expected
0 to 0.5 min	68	56.7
0.5 to 1.0 min	35	40.6
1.0 to 1.5 min	27	29.1
1.5 to 2.0 min	21	20.9
2.0 to 3.0 min	22	25.7
Over 3.0 min	27	27.1

Chi-square = 3.71, below the 5% cutoff of 9.49. We do not reject the Exponential model.

Does the Choice Actually Matter?

- All three service-time models have the same 1.5 minute mean
- The Exponential model gives mean wait 4.42 plus or minus 0.84 minutes
- The Uniform model gives mean wait 2.86 plus or minus 0.66 minutes
- The Constant model gives mean wait 2.43 plus or minus 0.51 minutes
- Occasional long services make the Exponential queue wait nearly twice as long
- Run demo 12_does_the_choice_matter.py after this slide

Key Takeaways

- Continuous simulation integrates a rate equation through time
- Euler's method uses the rate at the start of each step
- Step size affects both accuracy and stability
- Runge-Kutta obtains a better answer by evaluating the rate four times
- Input modelling uses observed shape and variability, not only the mean
- A fitted distribution needs an adequacy check and a decision-impact check

Thank you!